

HLSR: Hybrid Live–Forecast Selective Dynamic Vehicle Rerouting for Real-Time Congestion Avoidance

Xiao Wang, Shun-Ren Yang, *Member, IEEE*, and Hui-Nien Hung

Abstract—Urban traffic congestion reduces productivity and increases travel cost and emissions. Network-wide live travel-time shortest-path rerouting can be highly effective in simulation, but assumes that essentially every on-road vehicle is replanned every decision period. We propose HLSR, a *selective* hybrid live–forecast vehicle rerouting framework that fuses live edge speeds with short-horizon forecasts under limited intervention scope. Building on dual-threshold congestion detection, calibrated upstream selection, and driver-tailored travel-time prediction, HLSR further introduces approaching-vehicle expansion, travel-time-weighted k -shortest-path generation, and a horizon-dependent hybrid live–forecast segment speed used in multi-cost route allocation. On a reproduced Tainan SUMO scenario (seed 42), HLSR reduces mean travel time to 380.6 s at 8000 vehicles, outperforming same-scope live-only controls (438.1 s selective live; 391.6 s scoped live Dijkstra) and remaining competitive with network-wide live travel-time Dijkstra (408.1 s), while intervening on far fewer vehicles than a full-network policy; under the same protocol at 16000 and 20000 vehicles, HLSR remains best at 895.7 s and 971.7 s.

Index Terms—Traffic congestion detection, selective vehicle rerouting, hybrid live–forecast routing, travel-time prediction, SUMO

I. INTRODUCTION

The population gradually moves from rural areas to cities. A United Nations report [1] estimates that two-thirds of the world’s population will live in urban areas by 2050. Population growth increases traffic pressure, and road-capacity expansion alone is difficult to sustain because of cost and spatial constraints.

Current route navigation systems such as TomTom [2] and Google Maps [3] plan routes from infrastructure-based traffic information, but they cannot reliably replan vehicles that are already en route. Navigation without dynamic network-wide monitoring can even worsen congestion by funneling demand onto links with insufficient capacity. Intelligent transportation systems therefore combine connected sensing, edge/cloud computing, and data-driven control to mitigate congestion in real time.

Vehicle rerouting is a validated approach in this setting: when a segment becomes congested, the platform assigns alternative routes so that affected vehicles avoid the bottleneck and reduce travel time. Typical rerouting proceeds

in three steps—congestion detection, rerouted-vehicle selection, and alternative-route allocation—yet existing methods leave recurring limitations in each step. Single-indicator or locally scoped detectors often miss heterogeneous congestion patterns and network-wide coupling. Fixed spatial windows and greedy congestion-weighted selection tend to over-reroute distant vehicles or under-react to upstream traffic approaching a bottleneck. Route-allocation policies based only on current live traffic omit short-horizon forecast information needed for segments that will be traversed in later decision periods. Section II reviews representative prior work along these three dimensions.

To address these gaps, we introduce HLSR (Hybrid Live–Forecast Selective Rerouting), a selective congestion-alleviation framework. HLSR retains the three-phase structure above and strengthens it with hybrid live–forecast costing so that only congestion-related vehicles are replanned, while near-term decisions still react to observed speeds. The contributions of our work can be summarized as:

- **Proactive selective rerouting under bounded intervention:** HLSR integrates dual-threshold congestion detection, a calibrated upstream hop depth, and approaching-vehicle expansion so that rerouting remains limited to congestion-related vehicles. In the primary setting at 8000 vehicles, this selective stack reduces mean travel time from 664.2 s (BC) to 438.1 s (HLSR-LIVE).
- **Hybrid live–forecast route costing:** For each candidate route, HLSR blends live and predicted segment speeds with a horizon-dependent weight and applies travel-time-weighted k -shortest-path generation plus multi-cost allocation. Under matched selective candidate scope, this hybrid costing further improves travel time from 438.1 s (HLSR-LIVE) to 380.6 s (HLSR).
- **Driver-aware travel-time prediction module:** A network-level spatial–temporal forecaster (LSTAN_GERPE, optionally ranking-fine-tuned) supplies the forecast branch of the hybrid cost, with driver-behavior and footprint adjustments for personalization, rather than serving as a stand-alone end-to-end rerouter.
- **Fair-scope evaluation:** On the primary Tainan evaluation setting (SUMO seed 42) at 8000, 16000, and 20000 vehicles, we compare HLSR against classical selective baselines (NRR, ReFOCUS+), the reimplemented DuGAQ method, a prior HLSR-Rank checkpoint, same-scope live-only controls, and network-wide live travel-

X. Wang and S.-R. Yang are with the Department of Computer Science and the Institute of Communications Engineering, National Tsing Hua University, Hsinchu 30013, Taiwan (e-mail: sryang@cs.nthu.edu.tw).

H. Hung is with the Institute of Statistics, National Chiao Tung University, Hsinchu, Taiwan (e-mail: hhung@stat.nctu.edu.tw).

time Dijkstra, reporting travel time, time loss, waiting time, route length, and reroute count.

The remainder of this paper is organized as follows. Section II reviews prior rerouting methods by phase. Section III formalizes the system model and sensing quantities. Section IV details the travel-time prediction module that supplies multi-horizon predicted segment speeds V^{pred} . Section V assembles the HLSR selective rerouting loop, including hybrid live-forecast costing and the proactive extensions. Section VI reports the experimental settings and results. The paper concludes with limitations and future work.

II. RELATED WORK

Vehicle rerouting alleviates congestion by detecting bottlenecks, selecting vehicles to replan, and assigning alternative routes. The literature is organized below along these three phases, which also structure HLSR.

A. Congestion Detection

Proactive rerouting systems must declare congestion before downstream queues fully form. [4] couples detection with rerouting but relies on a single traffic-density indicator, which is insufficient when occupancy and speed diverge. [5] performs distributed detection through V2V cooperation, yet its single-vehicle velocity assumption and local neighborhood scope limit network-wide consistency. [6] combines segment-level and zone-level real-time scores, but aggregated indicators can still misclassify localized or transient congestion. Dual-threshold designs that jointly monitor occupancy and velocity, together with severity-aware triggers, are therefore preferable for selective intervention.

B. Rerouted-Vehicle Selection

Once congestion is detected, the controller must decide which vehicles to replan without flooding the network with unnecessary route changes. [7] weights all segments by congestion degree and greedily steers vehicles toward locally minimal-cost edges, which can inflate path length and shift congestion elsewhere. [8] randomly selects a fixed number of vehicles near a forecast congestion footprint, yet vehicles far from the bottleneck may be included while nearer upstream traffic is missed. A common alternative frames a spatial window around the congested segment and reroutes all vehicles inside it. [9] jointly optimizes rerouting and signal control but fixes the window size a priori; [10] broadcasts alarms through intelligent traffic lights with a fixed-level range; and [11] applies breadth-first search with a maximum depth, again using a static range. Fixed windows tend to over-reroute in mild congestion and under-reroute when severity grows, motivating a calibrated upstream hop depth together with approaching-vehicle expansion.

C. Alternative Route Allocation

The final phase assigns one or more candidate routes to each selected vehicle. Single-path policies risk shifting congestion from one area to another, so multi-path and multi-cost schemes

are widely adopted. [10] scores routes from current real-time network states, whereas [6] combines roadside-unit and traffic-management-center data through a Shannon-entropy router. Learning-based alternatives similarly emphasize live or region-level indices. For example, [12] couples fog-cloud GAQ with entropy-balanced k -shortest paths. Both families, however, rely primarily on *present* traffic conditions and do not explicitly fuse short-horizon forecasts with live sensing for segments that will be entered in future decision windows. Because road conditions evolve within each aggregation period, costing that blends near-term live speed with horizon-indexed predictions is needed for routes whose traversal extends beyond the current observation instant.

Collectively, prior selective rerouting methods improve individual phases but rarely integrate (i) dual-threshold detection, (ii) calibrated upstream and approaching-vehicle selection under bounded intervention, and (iii) hybrid live-forecast multi-cost allocation supported by a network-level forecaster. HLSR targets this integration gap while keeping forecasting as a supporting module rather than a stand-alone end-to-end router.

III. SYSTEM MODEL

This section first introduces the road-network representation and the zone partition under which HLSR operates. Second, the platform architecture and the sensing quantities maintained at each decision instant are presented. Third, we define the live and predicted speed values together with the travel-time components that later modules compose into route costs.

A. Road Network and Zone Partition

We consider a directed urban road network whose atomic elements are road segments (edges); let $\mathcal{S}$ denote the full set of segments. The network is partitioned into N_z city zones $\{z_1, z_2, \dots, z_{N_z}\}$. Without loss of generality, zone z_p contains N_{rs} road segments denoted $\{s_p^{(1)}, \dots, s_p^{(N_{rs})}\}$. Each segment s is characterized by length $L_{rs}(s)$, lane count $N_l(s)$, and speed limit $V^{lim}(s)$. Vehicles v report locations and destinations to the platform. τ denotes wall-clock time at a decision instant. The candidate-route budget N_{cr} is the maximum number of alternative paths retained for each replanned vehicle (default $N_{cr}=7$ in our deployment). In our scenario, vehicle arrivals and destinations are treated as an exogenous demand process independent of the selective controller, whereas congestion detection, vehicle selection, and hybrid route allocation constitute the joint controllable decision space optimized by HLSR.

B. Platform Architecture and Sensing

In Fig. 1, we demonstrate the cloud-hosted vehicle rerouting architecture that integrates roadside sensing with centralized selective control, wherein the transportation authority maintains a consistent view of segment occupancy and mean speed under periodic detector exports and vehicle location reports. In this architecture, roadside units periodically collect vehicle locations and destinations, while closed-circuit televisions and velocity / loop detectors export per-segment occupancy and

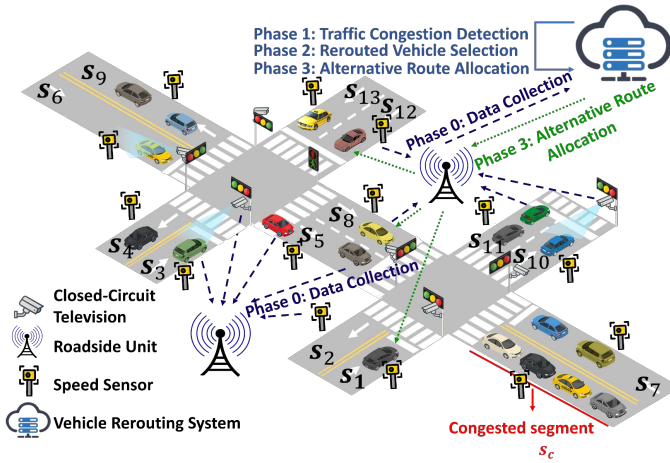


Fig. 1: Activity diagram of the traffic rerouting system.

mean-speed traces. Location and destination reports allow the platform to maintain the vehicle count $N_v(s, \tau)$ on each segment and to push updated routes to selected vehicles. Detector exports supervise the travel-time prediction module of Sec. IV and, at runtime, provide the live edge speeds consumed by hybrid costing in Sec. V.

C. Occupancy and Velocity Observables

From the maintained counts and speeds, the platform computes two normalized indicators that later enter congestion detection. Let $N_v(s, \tau)$ denote the number of vehicles on segment s at time τ . The road occupancy ratio is defined as

$$R_O(s, \tau) = \frac{N_v(s, \tau)}{N_v^{max}(s)}, \quad (1)$$

where the segment capacity is

$$N_v^{max}(s) = \frac{L_{rs}(s)}{\bar{L}_v + L_{sd}} \times N_l(s), \quad (2)$$

with average vehicle length $\bar{L}_v$ and safe spacing L_{sd} . The road velocity ratio is likewise defined as

$$R_V(s, \tau) = \frac{\bar{V}(s, \tau)}{V^{lim}(s)}, \quad (3)$$

where $\bar{V}(s, \tau)$ is the mean speed on s at τ . We model a discrete decision structure in which each aggregation window comprises an interval of length T (default $T=300$ s), indexed at successive window boundaries. Occupancy and speed statistics are accumulated over each window, and the selective control loop of Sec. V is invoked at those boundaries. Consequently, R_O and R_V furnish the measurable congestion state against which threshold-based detection operates, whereas route allocation itself is driven by the speed values introduced next.

D. Live and Predicted Speed Values

At decision time τ , live and predicted speed values are available for costing candidate routes. The live speed value $V^{live}(s, \tau)$ is the current mean speed on segment s obtained from live sensing (TraCI / detectors). The predicted speed

values $V^{pred}(s, \tau, h)$ are a multi-horizon forecast on s at horizon index $h=1, \dots, T_{out}$, where T_{out} is the forecast horizon length, produced by the travel-time prediction module of Sec. IV. HLSR never treats the predicted speed values as a stand-alone rerouter: Phase 3 of Sec. V blends V^{live} and V^{pred} with a horizon-dependent weight, wherein near-term edges place greater trust in live sensing while farther edges rely more on forecast foresight. Thus, live observations remain the authoritative near-horizon reference, whereas predicted speeds extend costing beyond the immediate sensing horizon without replacing selective control.

E. Travel-Time Components

When a vehicle v_r is scored on a candidate route $r_k(v_r, \tau) = \{s_k^{(1)} \rightarrow \dots \rightarrow s_k^{(\|r_k\|)}\}$, where $\|r_k\|$ denotes the number of segments on r_k , the segment travel time $t_v(s_k^{(j)}(v_r, \tau))$ decomposes into traffic-light queueing delay t_q and road-segment passing time t_p . The queueing model follows our earlier traffic-light formulation [13]. The passing time is obtained from a (hybrid) segment speed and, when enabled, a driver-tailored correction. Sec. IV specifies how V^{pred} and the driver factor are computed and illustrates the component breakdown. Sec. V specifies how those speeds are blended and embedded in multi-cost route allocation.

IV. TRAVEL TIME PREDICTION

This section specifies the travel-time prediction *module* that supplies multi-horizon predicted speed values $V^{pred}(s, \tau, h)$ for hybrid live-forecast route costing. It is not a stand-alone rerouting policy: the forecast is later blended with live speeds under a horizon-dependent weight, while selective vehicle selection remains separate. Building on the sensing metrics and travel-time components of Sec. III, we adopt our previously published spatio-temporal forecaster LSTAN_GERPE [14] to emit network-wide speed values. For the recommended HLSR configuration, the model is initialized from pretrained Huber weights and optionally fine-tuned by freezing the encoder and adapting only the output head with an additional path-ranking loss, so that candidate-route order is better preserved under route allocation. Driver-behavior personalization further maps the network-level forecast into vehicle-specific segment speeds for the selected candidates.

A. Forecasting Design

On forecasting v_r 's travel time $t_v(s_k^{(j)}(v_r, \tau))$ over segment $s_k^{(j)}(v_r, \tau)$ at a desired instant τ_j , a common practice is to predict edge speeds from recent local observations and convert those speeds into segment travel times. For selective hybrid rerouting, two issues matter more than edge-level regression accuracy alone. First, because HLSR scores full candidate routes rather than isolated sensors, forecast errors that preserve edge-level RMSE may still invert path rankings; the forecaster must therefore be trained and evaluated with route-level discrimination in mind. Second, since HLSR predicts t_v for a specific vehicle v_r , the driver's tendency to travel faster or slower than average also influences personalization accuracy. Taking these observations into account, we use a two-stage prediction stack:

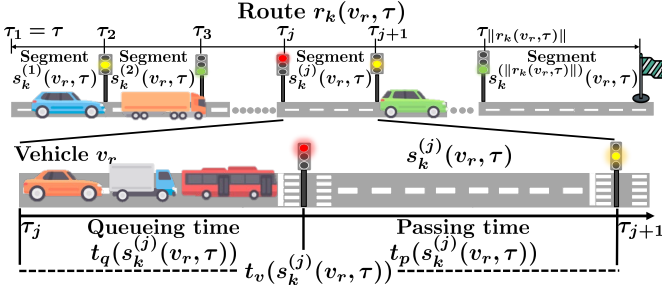


Fig. 2: Travel time component elements

1) *Network-level speed-value prediction*: A spatial-temporal forecaster (LSTAN_GERPE) consumes a short history of network-wide mean speed and occupancy and emits multi-horizon predicted speed values $V^{\text{pred}}(s, \tau, h)$ for every segment s and horizon index h . Daily recurring patterns are captured by GERPE pattern keys rather than by fitting a separate model per edge.

2) *Driver-tailored segment average velocity*: Given the network-level prediction for segment $s_k^{(j)}(v_r, \tau)$, a driver-behavior model maps the predicted speed into a vehicle-specific average velocity $V_A(\cdot)$ using the driver's behavior parameter $Bd(v_r, s)$ and a calibrated factor Δ . Together with a traffic-light delay model, this yields $t_v(s_k^{(j)}(v_r, \tau))$. The resulting personalized speed is consumed as the forecast branch of hybrid costing and does not replace live sensing for near-horizon segments.

B. The Prediction Framework

Given the current time τ as the entering time into the candidate shortest route $r_k(v_r, \tau)$, our framework operates iteratively as follows: provided with the estimated entering time τ_j into the road segment $s_k^{(j)}(v_r, \tau)$ obtained in the $(j-1)$ -th iteration, our framework predicts the travel time $t_v(s_k^{(j)}(v_r, \tau))$ over $s_k^{(j)}(v_r, \tau)$ and the entering time τ_{j+1} into the next road segment $s_k^{(j+1)}(v_r, \tau)$ in the j -th iteration. Fig. 2 illustrates the component elements of $t_v(s_k^{(j)}(v_r, \tau))$, which are further explained below.

1) *Traffic light queueing time* $t_q(s_k^{(j)}(v_r, \tau))$: This represents the delay that vehicle v_r should face at the entrance intersection of $s_k^{(j)}(v_r, \tau)$ due to a red traffic light upon the estimated entering time τ_j , which consists of 1) the remaining red-light time and 2) the transient residence time at the entrance intersection after the traffic light changes to green. Our framework applies the traffic light model we developed in [13] to estimate the two time segments, based on the pre-defined traffic-light signal timing plans (from the government transportation department) and a probabilistic model, respectively. The readers are referred to [13] for the details.

2) *Road segment passing time* $t_p(s_k^{(j)}(v_r, \tau))$: Let $\tau_{(j)}$ be $\tau_j + t_q(s_k^{(j)}(v_r, \tau))$, representing the instant when v_r leaves the traffic light and starts to cross $s_k^{(j)}(v_r, \tau)$. Let the horizon index aligned with $\tau_{(j)}$ be

$$h_j = \left\lfloor \frac{\tau_{(j)} - \tau}{T} \right\rfloor_+, \quad (4)$$

where T is the aggregation period of Sec. III and $[\cdot]_+$ clips negative values to zero. Our framework forecasts $t_p(s_k^{(j)}(v_r, \tau))$ as follows: 1) the network-level forecaster supplies $V^{\text{pred}}(s_k^{(j)}, \tau, h_j)$; 2) the driver-tailored model maps this speed to v_r 's average velocity $V_A(s_k^{(j)}(v_r, \tau), \tau_{(j)}, Bd(v_r, s_k^{(j)}))$ using the behavior parameter Bd introduced above; 3) finally,

$$t_p(s_k^{(j)}(v_r, \tau)) = \frac{L_{rs}(s_k^{(j)}(v_r, \tau))}{V_A(s_k^{(j)}(v_r, \tau), \tau_{(j)}, Bd(v_r, s_k^{(j)}))}.$$

The network-level forecaster and the driver-tailored model are elaborated in the following subsections.

C. Network-Level Spatio-Temporal Forecaster

This subsection details the forecasting module deployed within the HLSR framework. A per-edge LSTM stack is retained solely as an ablation baseline for comparative evaluation under the proposed HLSR pipeline and is not incorporated into our final recommended approach.

1) *Network-Wide Speed-Value Formulation*: Let $\mathcal{S}$ denote the set of road segments as in Sec. III, corresponding to nodes in the directed road graph, and $N = |\mathcal{S}|$ be its cardinality. At decision instant τ , measurements collected from loop detectors construct a historical input tensor

$$\mathbf{X}_\tau = \{x(s, \tau - i)\}_{s \in \mathcal{S}, i=0, \dots, T_{\text{in}}-1} \in \mathbb{R}^{T_{\text{in}} \times N \times F}, \quad (5)$$

where the input time window $T_{\text{in}} = 12$ (one-minute time steps), and feature dimension $F = 2$ stacks mean speed and occupancy. The forecasting model f_θ projects $\mathbf{X}_\tau$ toward multi-horizon segment-wise speed predictions:

$$V^{\text{pred}}(s, \tau, h) = f_\theta(\mathbf{X}_\tau; L_{\text{lap}})_{s,h}, \quad h = 1, \dots, T_{\text{out}}, \quad (6)$$

in which $T_{\text{out}} = 6$ defines the prediction horizon length, and L_{lap} denotes Laplacian positional encoding derived from the road graph.

During route assignment, $V^{\text{pred}}(s, \tau, h)$ is fused with real-time observed speeds in a horizon-adaptive manner. Near-future horizons prioritize live sensor measurements, whereas segments further ahead rely more heavily on model-predicted speed values. Different from per-edge sequential predictors that maintain an independent model instance for each detector, one single model forward pass produces globally consistent network-wide speed estimates shared by all candidate routing alternatives within the same decision cycle.

2) *LSTAN-GERPE Architecture*: The network-level forecaster adopts our previously published LSTAN_GERPE model [14], which we reuse as the speed-prediction backbone for HLSR. Briefly, it implements a graph-aware encoder-decoder architecture. Laplacian graph embedding, rotary positional encoding, and stacked spatio-temporal attention blocks jointly model geographic adjacency, semantic correlation among road segments, and temporal traffic evolution. The multi-horizon decoder outputs $V^{\text{pred}}(s, \tau, h)$ for $h = 1, \dots, T_{\text{out}}$.

Pretraining follows the same Huber regression objective as in our prior work [14]. Consequently, the edge-level Huber

TABLE I: Default hyperparameters of LSTAN_GERPE for speed-occupancy inputs, adopted from our prior work [14].

Parameter	Value
Input / output window ($T_{\text{in}}, T_{\text{out}}$)	12, 6
Features F	speed, occupancy
Embed / skip dimension	64 / 256
Encoder depth D	6
Geo / sem / temporal attention heads	4 / 2 / 2
Geographic hop mask Δ_{far}	7
Training loss	Huber ($\delta = 2$) [14]

loss $\mathcal{L}_{\text{Huber}}$ adopted herein inherits from that backbone and does not constitute a novel contribution of this paper. Table I summarizes hyperparameter settings when integrating this backbone into HLSR. Comprehensive architectural descriptions can be found in [14].

3) *Reroute-Aligned Fine-Tuning*: Pure edge-level regression accuracy cannot guarantee correct ranking of candidate routes under multi-objective route allocation. Even forecasts yielding acceptable Huber residuals may invert the relative travel-time ordering between complete paths, which consequently leads to sub-optimal route selections.

To mitigate this issue, we optionally fine-tune a ranking-aware variant denoted LSTAN_GERPE_rank. During fine-tuning, the encoder of our pretrained LSTAN_GERPE [14] is kept frozen; only the prediction output head is adapted via an OD-based k -shortest-path ranking loss. The composite training objective reads

$$\mathcal{L} = \mathcal{L}_{\text{Huber}} + \lambda_{\text{rank}} \mathcal{L}_{\text{od_ksp}}, \quad (7)$$

where $\mathcal{L}_{\text{Huber}}$ denotes the original backbone Huber loss for speed-value regression inherited from our prior work [14], and λ_{rank} is a small weighting coefficient. It softly regularizes route-order consistency without overriding the pretrained speed representations.

The ranking loss term is constructed to mimic the candidate-path comparison procedure within route allocation. For each origin-destination (OD) pair o encountered in selective rerouting demands from the SUMO simulator, we precompute a set $\mathcal{P}_o = \{P_{o,1}, \dots, P_{o,K}\}$ of K loop-free k -shortest paths [15] over the directed road graph, with $K = N_{\text{cr}}$ matching the candidate-route budget defined in Sec. III.

Given segment-wise speed values V , the travel time of a path P is computed by aggregating edge travel times:

$$T(P; V) = \sum_{e \in P} \frac{L_{\text{rs}}(e)}{\max(V(e), \varepsilon)}, \quad (8)$$

where $\varepsilon > 0$ is a small positive constant for numerical stability. By substituting the predicted speed values V^{pred} and ground-truth detector-measured speeds V^* into (8), we obtain predicted path travel times $\hat{T}(P)$ and ground-truth path travel times $T^*(P)$, respectively.

Within each OD candidate set $\mathcal{P}_o$, we sample path pairs (P_i, P_j) . Only pairs whose ground-truth time difference exceeds a threshold t_{min} are retained, such that near-tie samples do not dominate gradient updates. Let $\Delta T^* = T^*(P_i) - T^*(P_j)$ denote the ground-truth travel-time difference and

$\Delta \hat{T} = \hat{T}(P_i) - \hat{T}(P_j)$ denote its predicted counterpart. The pairwise logistic ranking penalty is formulated as

$$\ell(i, j) = \log \left(1 + \exp(-\text{sign}(\Delta T^*) \Delta \hat{T}) \right), \quad (9)$$

which approaches zero when predicted path ordering aligns with ground truth and increases as ranking inversion occurs.

Averaging (9) across sampled mini-batch OD pairs, valid filtered path pairs, and all prediction horizons yields the final ranking loss:

$$\mathcal{L}_{\text{od_ksp}} = \frac{1}{|\mathcal{H}| |\mathcal{O}|} \sum_{h \in \mathcal{H}} \sum_{o \in \mathcal{O}} \frac{1}{|\mathcal{Q}_o|} \sum_{(i,j) \in \mathcal{Q}_o} \ell_h(i, j), \quad (10)$$

where $\mathcal{O}$ is the collection of OD pairs within one mini-batch, $\mathcal{Q}_o$ stands for the filtered valid path subset of $\mathcal{P}_o$, and $\mathcal{H}$ represents the set of prediction horizons for V^{pred} . Since pairwise comparisons are constrained to k -shortest candidate paths sharing identical origin-destination pairs, $\mathcal{L}_{\text{od_ksp}}$ directly regularizes the path-discrimination task faced by route allocation, instead of performing arbitrary pairwise comparisons over isolated road segments or random graph walks.

Fine-tuning is performed in a closed loop with the selective rerouting simulator: updated forecasts are redeployed in HLSR, and the resulting traffic measurements supervise the next ranking update of the output head under a reduced learning rate. This refinement improves path-ranking discrimination while keeping the overall HLSR rerouting pipeline unchanged. Quantitative closed-loop comparisons among HLSR-LSTM, HLSR-Huber, and Rank-backed HLSR are reported in the experimental evaluation.

D. Driver-Tailored Segment Average Velocity

1) *Assumption and definition*: We take the assumption that we can get road segment s monitoring velocity and each vehicle v_r historical traffic information like velocity, entry time and exit time. Single vehicle average velocity $\bar{V}(s, \tau, v_r)$ is calculated as equation 11, where $t_r^{v_r}(\tau_x)$ and $t_r^{v_r}(\tau_y)$ are the measured exit time and entry time of vehicle v_r on road segment r from detailed traffic data, respectively. $L_{rs}(s)$ is the length of road segment s .

$$\bar{V}(s, \tau, v_r) = \frac{L_{rs}(s)}{t_r^{v_r}(\tau_x) - t_r^{v_r}(\tau_y)} \quad (11)$$

We consider driver's pattern which depends on many factors, such as automobile performance, driver's physiological condition and personal emotion, and express the complicated driver pattern as a simple driver behavior parameter $Bd(v, s)$ (Fig. 3). Behavior parameter $Bd(v_r, s)$ is calculated as equa-

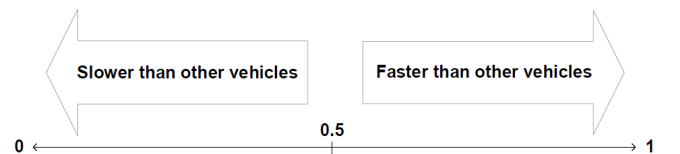


Fig. 3: Definition of the driver behavior parameter

tion 12.

$$Bd(v_r, s) = \frac{\bar{V}_{his}(s, v_r) - \bar{V}_{his}(s, v_N)}{\bar{V}_{his}(s, v_M) - \bar{V}_{his}(s, v_N)} \quad (12)$$

which is a Min-Max-Scaler normalization, where $\bar{V}_{his}(s, v_r)$ is historical average velocity of v_r , $\bar{V}_{his}(s, v_M)$ is historical average velocity of vehicle v_M with the maximum historical average velocity of all vehicles, $\bar{V}_{his}(s, v_N)$ is historical average velocity of vehicle v_N with the minimum historical average velocity of all vehicles.

The relationship between single vehicle average velocity $\bar{V}(s, \tau, v_r)$ and the network-level (predicted or live) segment speed $V(s, \tau)$ is expressed in equation 13.

$$\bar{V}(s, \tau, v_r) = V(s, \tau) \times \Delta \quad (13)$$

We define Δ as driver-tailored factor which is correlated to per driver behavior parameter $Bd(v_r, s)$.

2) *Determination of Δ* : Data acquisition and normalization. Based on traffic data and equation (11), the vehicles' average velocity is $\bar{V}_{set} = \{\bar{V}(s, \tau, v_r^0), \bar{V}(s, \tau, v_r^1), \dots, \bar{V}(s, \tau, v_r^{n_s-1})\}$, where n_s corresponds to the data size which stands for number of vehicles. Next, based on road detector information, we get the road instantaneous velocity named as $V_{set} = \{V(s, \tau, v_r^0), V(s, \tau, v_r^1), \dots, V(s, \tau, v_r^{n_s-1})\}$, where $V(s, \tau, v_r^{n_s-1})$ is the instantaneous velocity at time τ for road segment s where vehicle $v_r^{n_s-1}$ is exactly on. we use $z_{n_s-1}^s$ stands for $\frac{\bar{V}(s, \tau, v_r^{n_s-1})}{V(s, \tau, v_r^{n_s-1})}$. Next, we divide parallel elements in set $\bar{V}_{set}$ and V_{set} , gaining $Z_{set}^{ratio} = \{z_0^s, z_1^s, \dots, z_{n_s-1}^s\}$. Then, we normalize $z_j^s (j \in [0, n_s-1])$ to $z_j^{*s} \in (0, 1)$ by linear transformation shown in equation 14, improving prediction efficiency. The prediction performance works better for cases in which n_s is large enough.

$$\begin{aligned} z_j^{*s} &= \frac{z_j^s - z_{min}^s}{z_{max}^s - z_{min}^s} \times \frac{n_s - 1}{n_s} + \frac{1}{2 \times n_s}, \\ z_{max}^s &= \max_{j=0, \dots, n_s-1} z_j^s, \\ z_{min}^s &= \min_{j=0, \dots, n_s-1} z_j^s \end{aligned} \quad (14)$$

Next, we describe how we estimate Δ . We select fifty vehicles on each road segment and derive the related $z_j^s (j \in [0, 49])$. The observation layout and empirical CDFs used in calibration are reported with the prediction-module checks in the experimental evaluation. Let z denote one of the z_j^s values and n_z the number of occurrences of that value, so that $p(z^s) = n_z/50$. Because z^s is discrete, its cumulative distribution function is

$$F(z^s) = \sum_{Z \leq z^s} p(z^s). \quad (15)$$

Empirical CDFs of z^s on detector segments are close to a power function. We therefore approximate $F(z^s)$ by a parametric CDF of Δ and estimate the power via maximum likelihood [16]:

$$\eta_s = -\frac{n_s}{\sum_{j=0}^{n_s-1} \ln z_j^{*s}}. \quad (16)$$

Calculate $(Bd)^{\eta_s}$ and compress it in the x -direction by multiplying $(z_{max} - z_{min})$ to match the shape of $F(z^s)$, then shift right by adding z_{min} . As mentioned above, let $\hat{V}(s, \tau, v_r)$ (equivalently V_A) be the predicted average velocity of vehicle v_r on segment s at time τ , Bd the driver behavior parameter, and $V(s, \tau)$ the network-level segment speed (predicted speed values or live observation). The relationship between average velocity and segment speed is then

$$\begin{aligned} \hat{V}(s, \tau, v_r) &= V(s, \tau) \times [(z_{max}^s - z_{min}^s) \times (Bd)^{\eta_s} + z_{min}^s] \\ &= V(s, \tau) \times \Delta \end{aligned} \quad (17)$$

with

$$\Delta = [(z_{max}^s - z_{min}^s) \times (Bd)^{\eta_s} + z_{min}^s]. \quad (18)$$

Empirical agreement between $F(z)$ and $F(\Delta)$, together with the calibrated (z_{min}, z_{max}, η) values, is verified in the experimental evaluation.

3) *Time Cost Calculation*: Given the predicted segment speeds from Sec. IV-C and the driver factor Δ from (18), Algorithm 1 accumulates route travel time for candidate $r_k(v_r, \tau)$. For each segment $s_k^{(j)}$ on the route, the forecaster supplies $V^{\text{pred}}(s_k^{(j)}, \tau, h_j)$, the driver-tailored model yields $\hat{V}(s_k^{(j)}(v_r, \tau), \tau)$ via (17), and the segment time $L_{rs}/\hat{V}$ is added to the route cost.

Algorithm 1: $TC(f_\theta, \mathbf{X}_\tau, \{Bd, \eta, z_{max}, z_{min}, L_{rs}\})$

Input :
 Forecaster f_θ and history tensor $\mathbf{X}_\tau$;
 Per-segment parameters $\eta(s_k^{(j)})$, $z_{max}(s_k^{(j)})$, $z_{min}(s_k^{(j)})$,
 $Bd(v_r, s_k^{(j)})$, $L_{rs}(s_k^{(j)})$ for $j \in [1, \|r_k(v_r, \tau)\|]$;
Output :
 time cost $C_r^V(r_k(v_r, \tau))$;
Procedure :
 1 **Compute** the network-wide predicted speed values
 $V^{\text{pred}}(\cdot, \tau, \cdot) \leftarrow f_\theta(\mathbf{X}_\tau; L_{lap})$;
 2 $C_r^V(r_k(v_r, \tau)) = 0$;
 3 **for** $j = 1$; $j \leq \|r_k(v_r, \tau)\|$; $j++$ **do**
 4 $V(s_k^{(j)}, \tau) \leftarrow V^{\text{pred}}(s_k^{(j)}, \tau, h_j)$;
 5 **compute** $\hat{V}(s_k^{(j)}(v_r, \tau), \tau)$ **using** (17);
 6 $C_r^V(r_k(v_r, \tau)) = C_r^V(r_k(v_r, \tau)) + \frac{L_{rs}(s_k^{(j)}(v_r, \tau))}{\hat{V}(s_k^{(j)}(v_r, \tau), \tau)}$;
 7 **return** $C_r^V(r_k(v_r, \tau))$;

V. THE PROPOSED HLSR FRAMEWORK

Built upon the system model and travel-time forecaster from previous sections, this section details HLSR's selective real-time rerouting loop. Instead of recalculating paths for all vehicles per cycle, HLSR only replans vehicles impacted by congestion. We present its three-phase pipeline in order—congestion detection, vehicle selection, then route allocation—and finally assemble them into the main runtime loop. Core extensions include hybrid live-forecast costing, travel-time weighted k -shortest paths, and approaching-vehicle expansion. Fig. 4 visualizes data flow among live sensing, the prediction module, and the three functional phases.

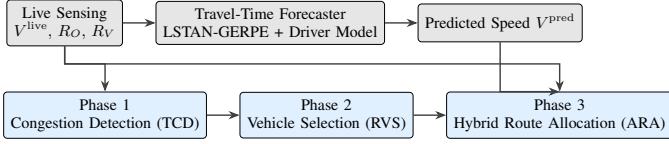


Fig. 4: HLSR runtime pipeline. Roadside sensing data feeds both the three-phase rerouting workflow and forecasting module. Phase 3 fuses live and predicted speeds to compute travel costs for candidate paths.

A. Phase 1: Traffic Congestion Detection

The core congestion detection logic is encapsulated in Algorithm 2. This module is structurally split into four standardized functional blocks: input specification, output definition, variable initialization, and execution procedure.

1) *Input Parameters*: The detection function accepts five mandatory arguments: 1) Target road segment s for congestion assessment; 2) Current decision timestamp τ ; 3) Threshold of occupancy ratio δ_O ; 4) Threshold of velocity ratio δ_V ; 5) Weight factor for occupancy-based congestion indicator Ψ_{ro} .

2) *Output Results*: Two outputs are returned after execution: a binary flag $isCongested$ indicating whether segment s is congested at time τ , and a continuous congestion severity metric $R_S(s, \tau)$.

3) *Initialization Stage*: Two internal variables are initialized before computation: $isCongested$ is set to False, and the severity score $R_S(s, \tau)$ is initialized to 0.

4) *Execution Procedure*: First, the module calculates normalized occupancy $R_O(s, \tau)$ and normalized velocity ratio $R_V(s, \tau)$ using the formulas derived in Sec. III. Congestion is triggered only when both traffic metrics exceed their respective thresholds, i.e., $R_O(s, \tau) > \delta_O$ and $1 - R_V(s, \tau) > \delta_V$. Once the dual-threshold condition is satisfied, $isCongested$ is assigned to True, and the composite congestion severity index is computed via:

$$R_S(s, \tau) = R_O(s, \tau) \cdot \Psi_{ro} + (1 - R_V(s, \tau)) \cdot (1 - \Psi_{ro}). \quad (19)$$

Finally, the flag $isCongested$ and severity value $R_S(s, \tau)$ are returned as function outputs. Collecting all congested segments at decision time τ yields the congested set

$$\mathcal{C}(\tau) = \{(s, R_S(s, \tau)) \mid s \in \mathcal{S}, isCongested(s, \tau)\}. \quad (20)$$

B. Phase 2: Rerouted Vehicle Selection

The vehicle screening logic is encapsulated in function RVS, as summarized in Algorithm 3. We break down its inputs, outputs, and computational workflow as follows.

1) *Input Parameters*: Three inputs are required for the selection module: 1) Congested target segment s_c ; 2) Current decision timestamp τ ; 3) Upstream hop depth θ_{ur} .

2) *Output Results*: The module outputs a vehicle set $S_{rv}^T(s_c, \tau)$, which collects all vehicles assigned to rerouting for mitigating congestion on s_c .

Algorithm 2: TCD: Dual-Threshold Traffic Congestion Detection Function

Input : Target road segment s ;
Current timestamp τ ;
Occupancy threshold δ_O ;
Velocity ratio threshold δ_V ;
Occupancy weight Ψ_{ro} ;

Output : Congestion flag $isCongested$;
Congestion severity $R_S(s, \tau)$;

Initialization: $isCongested \leftarrow \text{False}$;
 $R_S(s, \tau) \leftarrow 0$;

Procedure : Compute occupancy ratio $R_O(s, \tau)$ via (1) and (2);
Compute velocity ratio $R_V(s, \tau)$ via (3);

if
 $R_O(s, \tau) > \delta_O \wedge (1 - R_V(s, \tau)) > \delta_V$ **then**
 $isCongested \leftarrow \text{True}$;
Calculate $R_S(s, \tau)$ using (19);
return $isCongested, R_S(s, \tau)$;

3) *Execution Procedure*: We first define $S_{ur}^{(i)}(s_c, \tau)$ as the set of i -hop upstream segments of s_c at timestamp τ . These edges carry traffic flows that will enter s_c in subsequent aggregation cycles. For illustrative purposes, Fig. 1 gives an example: the 1-hop upstream set $S_{ur}^{(1)}(s_c, \tau) = \{s_1, s_5, s_8\}$ and the 2-hop upstream set $S_{ur}^{(2)}(s_c, \tau) = \{s_3, s_6, s_{12}, s_{13}\}$.

Let θ_{ur} denote the upstream hop depth, i.e., the maximum upstream layer considered for vehicle selection. A larger θ_{ur} expands the intervention range but also raises the risk of over-rerouting; the recommended deployment uses $\theta_{ur}=9$.

The full upstream segment pool is the union of hop layers $1, \dots, \theta_{ur}$:

$$S_{ur}^T(s_c, \tau) = \bigcup_{i=1}^{\theta_{ur}} S_{ur}^{(i)}(s_c, \tau). \quad (21)$$

The base candidate vehicle set consists of all vehicles residing within $S_{ur}^T(s_c, \tau)$ at timestamp τ :

$$S_{rv}^T(s_c, \tau) = \{v_r \mid v_r \text{ is located on any segment } s \in S_{ur}^T(s_c, \tau) \text{ at } \tau\}$$

a) *Approaching-Vehicle Expansion Mechanism*: Upstream selection and approaching expansion play complementary roles. The hop budget θ_{ur} sets a *topology* window $S_{ur}^T(s_c, \tau)$: vehicles currently located on those upstream segments form the base intervention set. Approaching expansion then supplements vehicles that lie *outside* this window yet will enter s_c shortly along their planned routes. Concretely, a vehicle is added if (i) s_c remains on its pre-planned route and (ii) the number of remaining route hops from its current edge to s_c is at most two (a fixed near-horizon budget). The final rerouting set for s_c is the union of the upstream base set and this approaching supplement; when a vehicle appears in both, the smaller hop index is kept for priority sorting.

Algorithm 3: RVS: Rerouted Vehicle Selection Function

Input : Congested road segment s_c ;
 Current decision timestamp τ ;
 Upstream hop depth θ_{ur} ;
Output : Aggregated rerouting vehicle set
 $S_{rv}^T(s_c, \tau)$ for segment s_c ;
Procedure: Aggregate full upstream segment pool
 $S_{ur}^T(s_c, \tau)$ using (21);
 Generate the base vehicle set from
 $S_{ur}^T(s_c, \tau)$ via (22);
 Expand with approaching vehicles outside
 that window whose remaining route hops to s_c are in
 $\{1, 2\}$;
 Return the union, keeping the smaller hop
 index per vehicle for later sorting;
1 return $S_{rv}^T(s_c, \tau)$;

C. Phase 3: Alternative Route Allocation

The route assignment logic is encapsulated in function ARA, whose full workflow is summarized in Algorithm 4. The input definitions, sorting strategy, hybrid live-forecast costing, candidate path generation, and multi-objective scoring are elaborated sequentially below.

1) *Input Parameters:* The allocation module accepts three mandatory inputs: 1. Current decision timestamp τ ; 2. Aggregated rerouting vehicle set $S_{rv}^T(\tau) = \bigcup_{(s_c, R_S) \in \mathcal{C}(\tau)} S_{rv}^T(s_c, \tau)$, merged from Phase 2 outputs over the congested set $\mathcal{C}(\tau)$ in (20); 3. Maximum number of candidate paths N_{cr} reserved for each individual vehicle.

2) *Execution Procedure:* For every vehicle requiring rerouting within $S_{rv}^T(\tau)$, the module generates N_{cr} feasible candidate paths and selects the optimal route via multi-objective cost minimization based on horizon-adaptive fused travel speeds.

a) *Horizon-Adaptive Live-Forecast Speed Fusion:* Let $V^{\text{live}}(s, \tau)$ denote real-time segment speed from roadside detectors, and $V^{\text{pred}}(s, \tau, h)$ denote personalized predicted speed output by the LSTAN-GERPE forecaster (integrated with driver-behavior calibration from Sec. IV). The proposed fusion strategy avoids over-reliance on pure prediction or raw sensing by assigning horizon-dependent blending coefficients: near-future segments trust live measurements, while far-ahead segments adopt forecasted traffic states:

$$\tilde{V}(s, \tau, h) = \alpha(h) V^{\text{live}}(s, \tau) + (1 - \alpha(h)) V^{\text{pred}}(s, \tau, h). \quad (23)$$

The live-data weight $\alpha(h)$ decays linearly with prediction horizon index h , with intercept α_0 and per-horizon decay α_Δ :

$$\alpha(h) = \max(0, \alpha_0 - h \cdot \alpha_\Delta). \quad (24)$$

For sufficiently large horizons satisfying $h \geq \lceil \alpha_0 / \alpha_\Delta \rceil$, $\alpha(h)$ drops to zero, and the fused speed fully relies on forecasting outputs. Real-time sensing acts as the authoritative reference for near-term traffic, while spatio-temporal forecasts extend cost calculation beyond the single aggregation window without triggering full-network vehicle replanning.

Notably, horizon index h does not equal the segment's sequential position on a route. Instead, h quantifies how many complete aggregation windows will elapse between the decision time τ and the vehicle's estimated entry instant t_{in} of the target segment:

$$h = \left\lfloor \frac{t_{\text{in}} - \tau}{T} \right\rfloor_+, \quad (25)$$

where T is the aggregation cycle of Sec. III, and $[\cdot]_+$ clips negative values to zero. The same physical road segment yields distinct h values for different vehicles due to divergent arrival timestamps, leading to unique fused speed weights per individual trajectory.

When evaluating a candidate path $r_k(v_r, \tau) = \{s_k^{(1)} \rightarrow s_k^{(2)} \rightarrow \dots \rightarrow s_k^{(\|r_k\|)}\}$, we iteratively compute entry time and horizon for each segment along the path: 1. Initialize entry time of the first segment $t_{\text{in}}^{(1)} \leftarrow \tau$; 2. For subsequent segment $s_k^{(j)}$, update arrival timestamp:

$$t_{\text{in}}^{(j)} = t_{\text{in}}^{(j-1)} + t_v(s_k^{(j-1)}), \quad (26)$$

where $t_v(s_k^{(j-1)})$ stands for the fused segment travel time (including optional traffic-light queue delay introduced in the travel-time prediction module). The corresponding horizon h_j for segment $s_k^{(j)}$ is obtained by substituting $t_{\text{in}}^{(j)}$ into (25), which matches (4) when $t_{\text{in}}^{(j)} = \tau_{(j)}$. Early route segments carry small h_j with dominant live-speed weight, whereas downstream long-distance segments use forecast-heavy fused speeds, matching the “near real-time, far predictive” design principle.

b) *Travel-Time Weighted Candidate Generation:* The raw vehicle set $S_{rv}^T(\tau)$ is first reordered into prioritized sequence $\tilde{S}_{rv}^T(\tau)$ by (i) ascending upstream hop index and (ii) descending residual destination distance, so that vehicles near bottlenecks with long residual trips are updated first. We then process each $v_r \in \tilde{S}_{rv}^T(\tau)$ sequentially and construct the candidate path set

$$S_{cr}(v_r, \tau) = \{r_k(v_r, \tau) \mid k = 1, 2, \dots, N_{cr}\}.$$

The urban road network is abstracted as a directed graph where vertices represent road segments and directed edges denote legal segment connections. To prioritize time-efficient corridors during candidate generation, we adopt Yen's K -shortest path algorithm [15] with dynamic travel-time edge weight

$$w(s, \tau) = \frac{L_{rs}(s)}{\tilde{V}(s, \tau, 0)}, \quad (27)$$

using the fused speed $\tilde{V}$ from (23) at the immediate horizon $h=0$. Unlike traditional length-weighted K -SP, this time-aware weighting scheme pushes fast, low-congestion corridors into the candidate pool before final multi-cost ranking.

c) *Multi-Objective Cost Allocation:* Given candidate path $r_k(v_r, \tau)$ with per-segment horizon h_j , the fused travel-time cost is accumulated as:

$$C_r^{tt}(r_k) = \sum_{j=1}^{\|r_k\|} \frac{L_{rs}(s_k^{(j)})}{\tilde{V}(s_k^{(j)}, \tau, h_j)}, \quad (28)$$

Algorithm 4: ARA: Adaptive Multi-Cost Alternative Route Allocation Function

Input : Current decision timestamp τ ;
Aggregated rerouting vehicle set $S_{rv}^T(\tau)$;
Maximum candidate route count N_{cr} ;
Procedure: $\hat{S}_{rv}^T(\tau) \leftarrow$ Sort $S_{rv}^T(\tau)$ by ascending upstream hop index, then descending residual destination distance;
foreach $v_r \in \hat{S}_{rv}^T(\tau)$ **in sorted order do**
Generate candidate set $S_{cr}(v_r, \tau)$ via travel-time weighted Yen K -SP using (27);
Compute normalized multi-cost scores and select optimal path $\hat{r}(v_r, \tau)$ via (23)–(29);
Send rerouting command to v_r for route $\hat{r}(v_r, \tau)$;

with optional additive traffic light waiting delay. HLSR jointly optimizes four normalized cost dimensions: travel time C^{tt} , path length C^d , route similarity penalty C^s , and network occupancy balance term C^o . All four metrics are min–max normalized over the candidate set $S_{cr}(v_r, \tau)$ to eliminate magnitude bias (denoted $\hat{C}^{tt}, \hat{C}^d, \hat{C}^s, \hat{C}^o$), and the optimal route is selected via weighted minimization with nonnegative weights w_t, w_d, w_s, w_o :

$$\hat{r}(v_r, \tau) = \arg \min_{r_k \in S_{cr}} (w_t \hat{C}^{tt} + w_d \hat{C}^d + w_s \hat{C}^s + w_o \hat{C}^o). \quad (29)$$

The recommended weight combination $(w_t, w_d, w_s, w_o) = (0.45, 0.20, 0.15, 0.20)$ prioritizes travel efficiency and short paths while introducing mild load balancing to disperse traffic pressure. Vehicles are rerouted sequentially; after each assignment, the platform updates predicted road occupancy footprints such that subsequent route selections account for newly diverted traffic flows. Finally, the platform issues route update instructions for vehicle v_r to switch to the optimal path $\hat{r}(v_r, \tau)$.

D. Main Runtime Loop

HLSR runs on a centralized cloud platform illustrated in Fig. 1 and co-simulates with the microscopic traffic simulator SUMO via the TraCI interface. During each fixed-length aggregation window, the platform continuously aggregates real-time segment speed and occupancy measurements collected from roadside detectors. Selective rerouting is triggered at each aggregation-window boundary of duration T (Sec. III). Algorithm 5 assembles Phases 1–3: at $\tau = nT$, TCD builds $\mathcal{C}(\tau)$ via (20); if nonempty, RVS unions per-bottleneck sets into $S_{rv}^T(\tau)$, and ARA assigns hybrid live–forecast routes using V^{pred} from Sec. IV.

The algorithm takes inputs $\mathcal{S}$, thresholds δ_O, δ_V , severity weight Ψ_{ro} , upstream depth θ_{ur} , candidate budget N_{cr} , and period T . After initialization, the main loop advances simulation time step-by-step. When $\tau = nT$, window-level statistics are aggregated and every $s \in \mathcal{S}$ is scanned by TCD. If $\mathcal{C}(\tau) \neq \emptyset$, Phase 2 merges upstream and approaching vehicles and Phase 3 performs one-shot hybrid route assignment.

Algorithm 5: Main Runtime Loop of the Proposed HLSR Framework

Input : The full set of road segments $\mathcal{S}$;
Occupancy threshold δ_O and velocity ratio threshold δ_V ;
Congestion severity weighting factor Ψ_{ro} ;
Upstream hop depth θ_{ur} ;
Maximum number of candidate routes N_{cr} ;
Aggregation cycle length T ;
Procedure: **while** *Unfinished trips remain in SUMO simulation* **do**
Advance simulation one time step and aggregate live segment speed / occupancy data; **if** τ aligns with window boundary $\tau = nT$ **then**
Aggregate window-wise traffic statistics over all $s \in \mathcal{S}$; $\mathcal{C}(\tau) \leftarrow \emptyset$; **foreach** $s \in \mathcal{S}$ **do**
 $isCongested, R_S(s, \tau) \leftarrow$
TCD($s, \tau, \delta_O, \delta_V$); **if** $isCongested$ **then**
 $\mathcal{C}(\tau) \leftarrow \mathcal{C}(\tau) \cup \{(s, R_S(s, \tau))\}$;
if $\mathcal{C}(\tau) \neq \emptyset$ **then**
 $S_{rv}^T(\tau) \leftarrow \bigcup_{(s, R_S) \in \mathcal{C}(\tau)} \text{RVS}(s, \tau, \theta_{ur})$;
ARA($\tau, S_{rv}^T(\tau), N_{cr}$) / * Trigger
forecast inference */
;

VI. PERFORMANCE EVALUATION

This section evaluates the proposed HLSR framework. We first ablate individual components and forecasters on the 8000-vehicle Tainan scenario, then compare HLSR with competing rerouting methods under 8000, 16000, and 20000 vehicles.

A. Simulation Settings

1) *Simulation Platform and Road Network:* HLSR is implemented in Python and co-simulated with Eclipse SUMO 1.18 [17], [18] via the TraCI interface, where the models presented in Secs. III–V are executed atop a microscopic traffic simulation engine. Deep-learning forecasting modules are trained using PyTorch 1.13.1.

Our simulation testbed covers the West Central District of Tainan. The road network is imported from OpenStreetMap and preprocessed with the Netconvert tool, yielding a topology consisting of 204 intersections, 561 road segments, and 77.3 km of total roadway length. The network is further divided into 14 traffic-analysis zones (TAZs). Among them, five zones serve as traffic origins ($Z_4, Z_{10}, Z_{12}, Z_{13}, Z_{14}$) and four act as destinations (Z_2, Z_3, Z_6, Z_9), which together construct 20 unique origin–destination (OD) pairs, as visualized in Fig. 5.

Vehicle travel demand is injected within a 7200 s simulation window and uniformly partitioned into four successive 1800 s intervals. Specifically, 25% of total vehicles are released in each time slot: $[0, 1800)$, $[1800, 3600)$, $[3600, 5400)$, and $[5400, 7200)$ s. Three traffic-demand scales, namely 8000,

16000, and 20000 vehicles, are evaluated on this fixed network topology. Each simulation trial runs until all generated vehicle trips are fully completed.

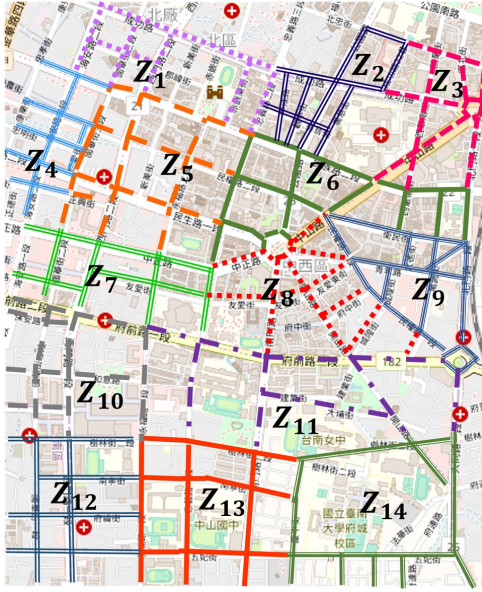


Fig. 5: Fourteen TAZs on the Tainan testbed (origins $Z_4, Z_{10}, Z_{12}, Z_{13}, Z_{14}$; destinations Z_2, Z_3, Z_6, Z_9).

2) *Evaluation Parameters:* All experiments use SUMO seed 42 and aggregation period $T=300$ s. The candidate-route budget is $N_{cr}=7$; thirty reduced-demand pilots showed that a larger budget does not further reduce mean travel time. On this protocol, recommended HLSR uses dual-threshold detection $(\delta_O, \delta_V)=(0.55, 0.45)$, hybrid blending $(\alpha_0, \alpha_\Delta)=(0.75, 0.12)$, multi-cost weights $(w_t, w_d, w_s, w_o)=(0.45, 0.20, 0.15, 0.20)$, and approaching expansion with two remaining route hops. Average vehicle length and safe spacing follow the SUMO defaults $\bar{L}_v=5$ m and $L_{sd}=2.5$ m.

The remaining stack parameter is the upstream hop depth θ_{ur} : a shallow window localizes intervention, whereas a deep window can over-reroute. Holding the settings above fixed, we sweep $\theta_{ur} \in \{1, \dots, 11\}$ at 8000 vehicles (Fig. 6). Mean travel time is high at $\theta_{ur}=1$ and 2, then nearly flat for $\theta_{ur} \in [3, 8]$ (384.4–390.5 s), while mean reroutes per vehicle increase almost monotonically. The minimum is at $\theta_{ur}=9$ (380.6 s), with $\theta_{ur}=10$ essentially tied (380.8 s); $\theta_{ur}=11$ rises to 388.0 s and the highest reroute rate (0.69). Subsequent HLSR rows in TABLES II–III therefore use $\theta_{ur}=9$. TABLE II also reports a shallower setting $\theta_{ur}=4$, which lies in the flat mid-range of Fig. 6 and is slightly worse.

3) *Travel-Time Prediction Setup:* The travel-time prediction module generates multi-horizon predicted speed outputs V^{pred} , which serve as the forecast inputs for hybrid cost computation in Phase 3. Loop detectors deployed across the Tainan road network (Fig. 7) output per-minute measurements of link-level speed and occupancy. These time-series observations are assembled into the historical input tensor $\mathbf{X}_\tau$ and drive the closed-loop fine-tuning procedure for the ranking-enhanced LSTAN_GERPE model.

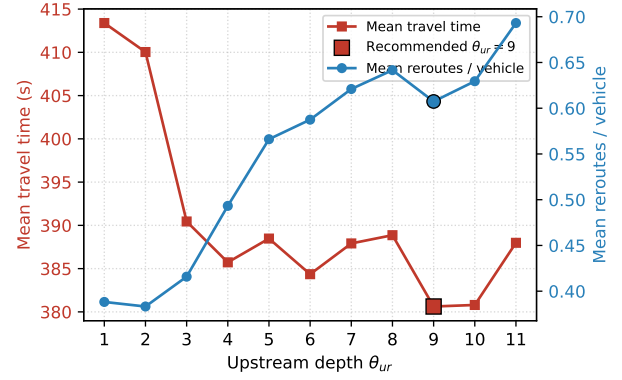


Fig. 6: Upstream-depth sweep on the recommended HLSR stack (8000 vehicles, SUMO seed 42). The highlighted marker is $\theta_{ur}=9$.

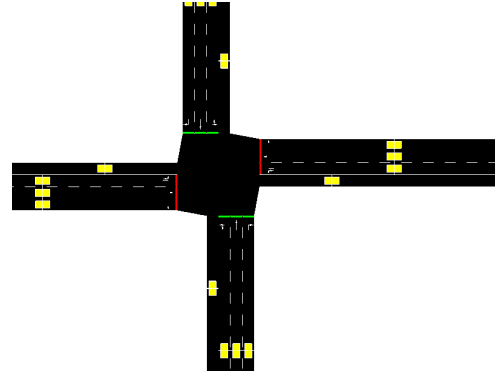


Fig. 7: Loop-detector placement on the Tainan SUMO network for per-minute edge-speed and occupancy export.

The network-wide forecasting backbone is pre-trained following the protocol described in our prior work [14]. For the recommended HLSR configuration, the encoder weights are frozen, and only the prediction head is fine-tuned with the OD-aware k -shortest-path ranking loss. The fine-tuning hyperparameters are set as: $\lambda_{\text{rank}}=0.015$, minimum path travel-time threshold $t_{\min}=10$ s, mini-batch size containing eight OD pairs (up to 32 candidate paths per OD pair), and learning rate 5×10^{-5} . A prototype per-edge LSTM model is preserved solely as an ablation baseline, corresponding to the HLSR-LSTM entry in TABLE II.

Driver-behavior parameters defined in (17) are calibrated using traffic traces collected from SUMO simulation and real-world detector records in Taipei. Empirical cumulative distribution functions (CDFs) of z_j^s validate the power-law formulation adopted for the driver-tailored factor Δ in (18). Fig. 8 provides a representative CDF comparison between empirical $F(z)$ and model-fitted $F(\Delta)$ for Nanjing Road; Civic Street and Xinyi Street exhibit consistent fitting performance.

B. Ablation Study

We quantitatively evaluate the individual contribution of each functional module within HLSR under the 8000-vehicle traffic scenario with SUMO seed 42. TABLE II summarizes

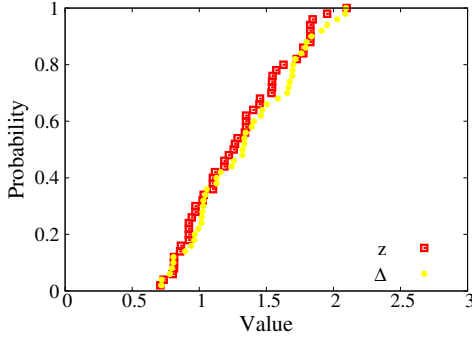


Fig. 8: Representative fit of model $F(\Delta)$ to empirical $F(z)$ (Nanjing Road; Civic and Xinyi show the same pattern).

the mean travel-time and average reroutes-per-vehicle metrics, with all results reported relative to the full recommended HLSR configuration ($\theta_{ur}=9$, Rank-enhanced forecasting). Block I performs leave-one-component-out ablation, where one controller module is removed or modified in each trial. Block II keeps the full HLSR control pipeline intact and only replaces the Phase-3 travel-time forecaster V^{pred} . A positive value of Δ indicates degraded performance, i.e., a higher mean travel time compared with the baseline HLSR.

1) *HLSR Component Ablation*: Disabling the hybrid live-forecast costing mechanism yields the largest performance degradation in Block I, corresponding to the HLSR-LIVE case with a +57.5 s travel-time penalty. Since the set of candidate rerouting vehicles remains identical across this comparison, this performance gap originates from the Phase-3 speed fusion logic rather than the upstream vehicle-selection module. Time-exclusive cost scoring (HLSR-time) and occupancy-only congestion detection (HLSR w/o R_V) increase mean travel time by 25.0 s and 19.7 s, respectively. These observations demonstrate that both multi-objective cost allocation and the velocity-ratio congestion threshold bring tangible benefits once hybrid costing is enabled. Removing driver-behavior personalization (HLSR-noDB) incurs a moderate yet consistent penalty of 12.4 s. Reducing the upstream hop depth from $\theta_{ur}=9$ to $\theta_{ur}=4$ (HLSR-fixed θ) raises the mean travel time to 385.7 s (+5.1 s), which aligns with the flat performance plateau observed in Fig. 6.

2) *Forecaster Comparison*: Block II isolates the impact of different Phase-3 prediction backbones while preserving the complete HLSR control framework. HLSR-LSTM adopts a per-edge LSTM predictor. HLSR-Huber utilizes the network-level LSTAN_GERPE backbone introduced in Sec. IV-C, but omits path-ranking fine-tuning. By contrast, the recommended HLSR employs the identical backbone enhanced with OD-aware k -shortest-path ranking loss described in Sec. IV-C3. Compared against the baseline HLSR (380.6 s), HLSR-LSTM and HLSR-Huber degrade by 7.6 s and 20.2 s, respectively. This result validates that ranking-aware fine-tuning constitutes the preferred output-head configuration for the network-level forecaster; meanwhile, the per-edge LSTM achieves substantially better performance than the Huber-only variant. Nevertheless, the performance gaps intro-

duced by different forecasting backbones are smaller than the large penalty observed for HLSR-LIVE. This confirms that hybrid live-forecast costing serves as the primary source of overall performance improvement.

C. Baseline Routing Comparison

We compare recommended HLSR with competing rerouting methods under the same seed 42 protocol, extending the 8000-vehicle setting of TABLE II to 16000 and 20000 vehicles. TABLE III reports mean travel duration and reroutes per vehicle; each cell is one complete trial on a fixed demand file. The compared methods are partially reimplemented on our TraCI platform while preserving the core selection and costing mechanisms documented in the source papers.

- 1) **NRR** [10]: congestion is indicated by road occupancy, upstream depth is fixed at $\theta_{ur} = 1$, and multi-factor next-edge costs determine the immediate successor segment.
- 2) **ReFOCUS+** [6]: congestion is declared from averaged occupancy and velocity scores, upstream depth is fixed at $\theta_{ur} = 3$, and Shannon-entropy routing is executed under traffic-management-center control.
- 3) **Du-GAQ** [12]: fog-cloud GAQ region indices with entropy-balanced k -shortest-path (EBkSP) assignment (Priority-Near; Tainan-adapted GAQ checkpoint).
- 4) **HLSR-Rank (classic selective)**: an earlier selective checkpoint that uses Rank forecasts without the full recommended OPT control stack.
- 5) **HLSR-LIVE**: the candidate vehicle set of HLSR is retained, whereas Phase-3 costing uses live speeds only.
- 6) **CAIE-TT-Scoped** [19]: live travel-time Dijkstra is applied exclusively to the same selective candidate set.
- 7) **CAIE-TT** [19]: live travel-time Dijkstra is applied to essentially all on-road vehicles every decision period, thereby embodying a stronger network-wide deployment assumption.

These baselines therefore span classical and region-based selective rerouting (NRR, ReFOCUS+, Du-GAQ, HLSR-Rank), live-only control on the HLSR candidate set (HLSR-LIVE, CAIE-TT-Scoped), and network-wide live travel-time Dijkstra (CAIE-TT). HLSR-LIVE is both the Block I ablation of hybrid costing and the fair-scope live baseline in this comparison.

a) *Travel-Time and Scope Analysis*: At 8000 vehicles, we first hold the OPT candidate set fixed. HLSR attains 380.6 s, improving upon HLSR-LIVE (438.1 s) by 57.5 s and upon CAIE-TT-Scoped (391.6 s) by 11.0 s. Hybrid forecast information is therefore beneficial when vehicle selection and live sensing are aligned. Network-wide CAIE-TT remains slower (408.1 s) despite replanning far more vehicles, so a larger intervention set does not replace hybrid costing. Among the remaining selective methods, Du-GAQ is the strongest at this demand (425.5 s) yet still trails HLSR by 44.9 s, while NRR, ReFOCUS+, and HLSR-Rank remain above 525 s. HLSR also attains the lowest time loss (215.5 s) and waiting time (150.1 s) among the compared methods.

b) *Multi-Demand Comparison*: Mean travel times increase with demand, yet HLSR remains best at every level in

TABLE II: Ablation study on the 8000-vehicle Tainan scenario under SUMO seed 42. Block I varies one controller component; Block II varies only the Phase-3 forecaster. Differences are relative to recommended HLSR (Rank forecasting; positive = worse).

Configuration	Travel time		Reroutes per vehicle	
	mean (s)	Δ vs HLSR	mean	Δ vs HLSR
<i>Block I: HLSR-anchored component ablation</i>				
HLSR	380.6	—	0.61	—
HLSR w/o R_V (EC1)	400.3	+19.7	0.45	−0.16
HLSR-fixed θ (EC2)	385.7	+5.1	0.49	−0.12
HLSR-LIVE (EC3)	438.1	+57.5	0.80	+0.19
HLSR-time (EC4)	405.6	+25.0	0.56	−0.05
HLSR-noDB	393.0	+12.4	0.51	−0.10
<i>Block II: forecaster ablation on the HLSR stack</i>				
HLSR-LSTM (per-edge)	388.2	+7.6	0.49	−0.12
HLSR-Huber	400.8	+20.2	0.56	−0.05

TABLE III: Baseline comparison across demand levels (SUMO seed 42). Entries report mean travel duration (s) and mean reroutes per vehicle (RR). **HLSR** denotes the recommended hybrid selective configuration. At 8000 vehicles, HLSR also attains time loss 215.5 s and waiting time 150.1 s (lowest among compared methods).

Method	Scope	8000		16000		20000	
		Dur.	RR	Dur.	RR	Dur.	RR
NRR	$\theta_{ur}=1$	525.5	0.36	1211.5	1.36	1276.9	1.42
ReFOCUS+	$\theta_{ur}=3$	632.1	0.84	1079.8	1.96	1151.5	2.20
HLSR-Rank (classic selective)	selective	609.7	0.68	1113.7	1.57	1102.4	1.58
Du-GAQ	GAQ+EBkSP	425.5	0.64	1384.4	3.84	1463.1	4.09
HLSR-LIVE	OPT selection	438.1	0.80	1281.3	6.36	1410.0	7.26
CAIE-TT	all vehicles	408.1	0.48	1013.9	2.30	1180.0	2.74
CAIE-TT-Scoped	OPT selection	391.6	0.35	1011.6	2.53	1120.3	2.84
HLSR	OPT selection	380.6	0.61	895.7	3.62	971.7	4.06

TABLE III (895.7 s / 971.7 s at 16000 / 20000 vehicles). The same-scope gaps widen: CAIE-TT-Scoped trails by 115.9 s and 148.6 s, and HLSR-LIVE degrades to 1281.3 s / 1410.0 s with reroute rates of 6.36 / 7.26. Du-GAQ, which was competitive at 8000 vehicles, likewise degrades to 1384.4 s / 1463.1 s (gaps of 488.7 s / 491.4 s). NRR, ReFOCUS+, and HLSR-Rank remain substantially slower at both higher demands, and network-wide CAIE-TT still trails HLSR (1013.9 s / 1180.0 s). Relative to the 8000-vehicle no-reroute floor (NONE, 1387.1 s), HLSR reduces mean travel time by 72.6% while keeping mean reroutes well below the LIVE extreme.

D. Discussion

Results in TABLES II and III indicate that HLSR’s travel-time reduction arises from coupled multi-module contributions instead of one dominant component. Fixing the OPT candidate set, disabling hybrid live-forecast costing (HLSR-LIVE) produces the largest penalty (+57.5 s) and yields inferior performance across all demand levels. HLSR outperforms CAIE-TT-Scoped by 11.0 s at 8000 vehicles and surpasses network-wide CAIE-TT despite operating with a much smaller rerouting set, verifying that Phase-3 live-forecast fusion serves as the primary performance driver.

Under hybrid costing, other components contribute in descending order: time-only scoring (+25.0 s), occupancy-only detection (+19.7 s), disabled driver personalization (+12.4 s), and shallower upstream depth $\theta_{ur}=4$ (+5.1 s). Block II demonstrates that ranking-aware forecasting is preferred;

the performance gaps from HLSR-LSTM (+7.6 s) and HLSR-Huber (+20.2 s) are both smaller than the HLSR-LIVE penalty. Therefore, the forecaster supports hybrid-cost calculation rather than replacing the rerouting controller.

HLSR retains state-of-the-art performance at 16000 and 20000 vehicles (895.7 s, 971.7 s). Baseline gaps widen under heavy congestion, where HLSR-LIVE suffers severe over-rerouting. Classical selective approaches remain slower, and Du-GAQ does not scale well to high-demand cases. In summary, HLSR ($\theta_{ur}=9$, Rank forecasting) yields consistent travel-time gains over competing baselines without excessive replanning overhead.

VII. CONCLUSION

This paper presents HLSR, a selective hybrid live-forecast vehicle rerouting framework designed for real-time urban congestion mitigation. In contrast to network-wide live shortest-path rerouting schemes that recompute routes for nearly all in-traffic vehicles at every decision step, HLSR constrains rerouting interventions to only congestion-affected vehicles. It computes alternative routes by fusing real-time link-level speed observations with short-horizon traffic forecasts.

The proposed framework integrates four key functional components: dual-threshold occupancy-velocity congestion detection, calibrated upstream hop depth together with approaching-vehicle expansion, travel-time-weighted k -shortest-path candidate generation, and horizon-adaptive

hybrid costing for multi-objective route allocation. A network-level spatio-temporal forecaster provides the predicted speed outputs V^{pred} for the hybrid-cost computation; importantly, this forecasting module serves purely as a supporting component rather than functioning as a standalone end-to-end rerouting solution.

Evaluated on the reproduced Tainan SUMO simulation testbed, the recommended HLSR configuration achieves lower mean travel time compared with fair-scope live-only baselines, and outperforms network-wide live Dijkstra while operating with a considerably smaller set of rerouted vehicles. Ablation-study results demonstrate that hybrid live-forecast costing contributes the major portion of performance improvement, while the specific choice of the prediction backbone for V^{pred} plays a secondary supporting role. This performance hierarchy holds consistently under elevated traffic demand: HLSR maintains the best overall performance across all evaluated load levels, with growing performance margins against same-scope live-only approaches, which suffer from severe over-rerouting under heavy congestion.

REFERENCES

- [1] U. Nations, *World Urbanization Prospects*. United Nations, 2014.
- [2] "Tomtom [online]." Available at <https://www.tomtom.com/>.
- [3] "Google maps [online]." Available at <https://www.google.com/mobile/maps/>.
- [4] J. Pan, M. A. Khan, I. S. Popa, K. Zeitouni, and C. Borcea, "Proactive vehicle re-routing strategies for congestion avoidance," in *2012 IEEE 8th International Conference on Distributed Computing in Sensor Systems*, pp. 265–272, IEEE, 2012.
- [5] M. Milojevic and V. Rakocevic, "Short paper: Distributed vehicular traffic congestion detection algorithm for urban environments," in *2013 IEEE Vehicular Networking Conference*, pp. 182–185, 2013.
- [6] M. Rezaei, H. Noori, M. Mohammadkhani Razlighi, and M. Nickray, "Refocus+: Multi-layers real-time intelligent route guidance system with congestion detection and avoidance," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 1, pp. 50–63, 2021.
- [7] A. M. de Souza, R. S. Yokoyama, L. C. Botega, R. I. Meneguette, and L. A. Villas, "Scorpion: A solution using cooperative rerouting to prevent congestion and improve traffic condition," in *2015 IEEE International Conference on Computer and Information Technology; Ubiquitous Computing and Communications; Dependable, Autonomic and Secure Computing; Pervasive Intelligence and Computing*, pp. 497–503, 2015.
- [8] C. Backfrieder, G. Ostermayer, and C. F. Mecklenbräuker, "Increased traffic flow through node-based bottleneck prediction and v2x communication," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 2, pp. 349–363, 2017.
- [9] Z. Cao, S. Jiang, J. Zhang, and H. Guo, "A unified framework for vehicle rerouting and traffic light control to reduce traffic congestion," *IEEE Transactions on Intelligent Transportation Systems*, vol. 18, no. 7, pp. 1958–1973, 2017.
- [10] S. Wang, S. Djahel, Z. Zhang, and J. McManis, "Next road rerouting: A multiagent system for mitigating unexpected urban traffic congestion," *IEEE Transactions on Intelligent Transportation Systems*, vol. 17, no. 10, pp. 2888–2899, 2016.
- [11] M. Rezaei, H. Noori, D. Rahbari, and M. Nickray, "Refocus: A hybrid fog-cloud based intelligent traffic re-routing system," in *2017 IEEE 4th International Conference on Knowledge-Based Engineering and Innovation (KBEI)*, pp. 0992–0998, 2017.
- [12] R. Du, S. Chen, J. Dong, T. Chen, X. Fu, and S. Labi, "Dynamic urban traffic rerouting with fog-cloud reinforcement learning," *Computer-Aided Civil and Infrastructure Engineering*, vol. 39, no. 6, pp. 793–813, 2024.
- [13] S.-R. Yang, Y.-J. Su, Y.-Y. Chang, and H.-N. Hung, "Short-term traffic prediction for edge computing-enhanced autonomous and connected cars," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 4, pp. 3140–3153, 2019.
- [14] X. Wang and S.-R. Yang, "Lightweight spatio-temporal attention network with graph embedding and rotational position encoding for traffic forecasting," in *2025 IEEE International Conference on Service Operations and Logistics, and Informatics (SOLI)*, pp. 1–6, 2025.
- [15] J. Y. Yen, "Finding the k shortest loopless paths in a network," *Management Science*, vol. 17, no. 11, pp. 712–716, 1971.
- [16] H. Akaike, "Information theory and an extension of the maximum likelihood principle," in *Selected papers of hirotugu akaike*, pp. 199–213, Springer, 1998.
- [17] M. Behrisch, L. Bieker, J. Erdmann, and D. Krajzewicz, "Sumo – simulation of urban mobility: An overview," in *SIMUL 2011* (S. . U. of Oslo Aida Omerovic, R. I. R. T. P. D. A. Simoni, and R. I. R. T. P. G. Bobashev, eds.), ThinkMind, Oktober 2011.
- [18] D. Krajzewicz, J. Erdmann, M. Behrisch, and L. Bieker, "Recent development and applications of sumo-simulation of urban mobility," *International journal on advances in systems and measurements*, vol. 5, no. 3&4, 2012.
- [19] D. Włodarczyk and T. Saber, "Uav-assisted traffic rerouting in disaster scenarios via grammar-guided genetic programming: Effects of limited smart-vehicle adoption and cross-segment generalization," *Computers & Industrial Engineering*, vol. 215, p. 111839, 2026.

A. Notation List

TABLE IV: Notations (editing reference; remove before submission)

Symbol	Description
L_{sd}	Safe distance between two consecutive vehicles.
$L_{rs}(s)$	Length of road segment s .
$\bar{L}_v$	Average vehicle length.
N_{cr}	Maximum number of candidate shortest routes per replanned vehicle.
N_{rs}	Number of road segments in city zone z_p (equal-count WLOG).
$N_v(s, \tau)$	Number of vehicles on segment s at time τ .
$N_v^{max}(s)$	Capacity of segment s (vehicles).
$N_l(s)$	Number of lanes on segment s .
N_z	Number of city zones.
$R_O(s, \tau)$	Road occupancy ratio on s at τ (1).
$R_V(s, \tau)$	Road velocity ratio on s at τ (3).
$R_S(s, \tau)$	Congestion severity of s at τ (19).
$r_k(v_r, \tau)$	k -th candidate route for rerouted vehicle v_r at τ .
$\ r_k\ $	Number of segments on candidate route r_k .
$\hat{r}(v_r, \tau)$	Selected alternative route for v_r at τ (29).
$s_p^{(q)}$	q -th road segment of city zone z_p .
$s_k^{(j)}(v_r, \tau)$	j -th segment on candidate route $r_k(v_r, \tau)$.
$S_{ur}^{(i)}(s_c, \tau)$	Set of i -hop upstream segments of congested segment s_c .
$S_{ur}^T(s_c, \tau)$	Union of upstream layers $1, \dots, \theta_{ur}$ for s_c (21).
$S_{rv}^T(s_c, \tau)$	Vehicles selected for rerouting w.r.t. s_c (22).
$S_{cr}(v_r, \tau)$	Set of N_{cr} candidate routes for v_r .
$\bar{V}(s, \tau)$	Mean speed on segment s at τ .
$\hat{V}(s, \tau, v_r)$	Driver-tailored predicted average velocity of v_r on s .
$V^{lim}(s)$	Speed limit on segment s .
z_p	p -th city zone.

Continued on next page

TABLE IV: Notations (editing reference; remove before submission) (Continued)

Symbol	Description
δ_O, δ_V	Occupancy / velocity-ratio thresholds for dual-threshold TCD.
Ψ_{ro}	Weight of R_O in severity R_S (19).
θ_{ur}	Upstream hop depth for vehicle selection (recommended $\theta_{ur}=9$).
$\tilde{V}(s, \tau, h)$	Hybrid live–forecast speed at horizon h (23).
$V^{live}(s, \tau), V^{pred}(s, \tau, h)$	Live and predicted segment speeds in (23).
$\alpha(h)$	Live-speed blending weight at horizon h (24).
α_0, α_Δ	Intercept and per-horizon decay of $\alpha(h)$.
$t_{in}^{(j)}$	Estimated entry time into the j -th edge of a candidate route (26).
$\mathcal{C}(\tau)$	Congested segment set at decision time τ (20).
h_j	Horizon index aligned with segment entry time (4, 25).
$\hat{C}^{tt}, \hat{C}^d, \hat{C}^s, \hat{C}^o$	Min–max normalized multi-cost terms in (29).
$\tilde{S}_{rv}^T(\tau)$	Priority-sorted copy of $S_{rv}^T(\tau)$ for sequential ARA.
$\mathcal{S}$	Full set of road segments.
T, T_{out}	Aggregation period; forecast horizon length.
$Bd(v_r, s), \Delta, \eta_s, z_{max}^s, z_{min}^s$	Driver-behavior parameter and calibration factors (Sec. IV-D).
$V_A(\cdot)$	Driver-tailored segment speed; realized as $\hat{V}$ in (17).
$C_r^{tt}(r_k)$	Fused travel-time cost of candidate r_k (28).
$\mathbf{X}_\tau$	Network-wide history tensor of speed and occupancy over T_{in} steps.
f_θ	Network-level spatio-temporal forecaster (LSTAN_GERPE / Rank).
$\mathcal{L}_{Huber}$	Backbone Huber speed-value loss.
$\mathcal{L}_{od_ksp}$	OD– k -shortest-path pairwise ranking loss (10).
$T(P; V)$	Path travel time of candidate P under speeds V (8).
λ_{rank}	Weight of $\mathcal{L}_{od_ksp}$ in Rank fine-tuning.

Continued on next page

TABLE IV: Notations (editing reference; remove before submission) (Continued)

Symbol	Description
Approaching route-hop budget	Fixed near-horizon budget (default 2) for route-based complement to θ_{ur} .
w_t, w_d, w_s, w_o	Multi-cost weights for travel time, distance, similarity, occupancy.
$C_r^T(\cdot)$	<i>[obsolete]</i> Old total-route cost; use C_r^{tt} / (29).
$L_{rs}^{max}, L_{rs}^{min}$	<i>[obsolete]</i> Longest/shortest segment lengths; unused.
$R_L(s)$	<i>[obsolete]</i> Normalized segment length; unused.
$R_O^z(z(s), \tau)$	<i>[obsolete]</i> Zone occupancy ratio; unused.
$\Psi_{cr}(s, \tau)$	<i>[obsolete]</i> Old K-SP edge weight; use (27).